StudySync

Project Documentation

An AI-powered, multi-platform system that reads exam papers with OCR and explains every answer step by step on web, desktop and mobile.

VERSION	**1.0**
DATE	**10 October 2026**
PREPARED BY	**JozConsortia**
CONTACT	**josephmokhampane2@gmail.com**

Web · Desktop · Mobile · Python FastAPI · MySQL · OCR · OpenAI

Contents

1. Introduction . . . 4
1.1 Purpose of this document . . . 4
1.2 The problem . . . 4
1.3 Objectives . . . 4
1.4 Scope . . . 5
1.5 Definitions . . . 5
2. Features . . . 6
3. System architecture . . . 7
3.1 Components . . . 7
3.2 Design decisions . . . 8
3.3 Technology stack . . . 8
4. How it works . . . 9
4.1 Extracting questions . . . 9
4.2 Solving a question . . . 10
4.3 History and saved answers . . . 10
5. Database design . . . 11
5.1 users . . . 11
5.2 history_sessions . . . 12
5.3 history_questions . . . 12
5.4 saved_answers . . . 13
5.5 Indexes and integrity . . . 13
6. Backend API reference . . . 14
6.1 Endpoints . . . 14
6.2 Example: solving a question . . . 15
6.3 Example: saving a history session . . . 15
6.4 Errors . . . 15
6.5 Validation limits . . . 16
7. The applications . . . 17
7.1 Web app . . . 17
7.2 Desktop app . . . 19
7.3 Mobile app . . . 20
8. Security . . . 21

9. Installation and configuration . . . 22
9.1 Requirements . . . 22
9.2 Install . . . 22
9.3 Configure the server . . . 22
9.4 Create the database . . . 23
9.5 Run . . . 23
9.6 Pointing the apps at the API . . . 23
10. User guide . . . 24
10.1 Create an account and sign in . . . 24
10.2 Scan a paper . . . 24
10.3 Get solutions . . . 24
10.4 Save and download . . . 24
10.5 Use your history . . . 25
11. Testing . . . 26
11.1 OCR . . . 26
11.2 API and database . . . 26
11.3 End-to-end . . . 26
12. Limitations and future work . . . 28
12.1 Known limitations . . . 28
12.2 Future work . . . 28
13. Troubleshooting . . . 29
Appendix A. Project structure . . . 30
A.1 Environment variables (server/.env) . . . 30
A.2 npm scripts (project root) . . . 31

1. Introduction

StudySync is a multi-platform, AI-powered study system. A student uploads an examination paper as a PDF or an image, StudySync reads every question with Optical Character Recognition (OCR), and OpenAI generates a solution with a step-by-step explanation for each question. Students review, correct, save and download the answers, and everything they do is kept in their account so they can continue on the web, on the desktop or on their phone.

1.1 Purpose of this document

This document describes StudySync as built: what it does, how it is designed, how the parts communicate, how data is stored, how to install and run it, and how it was tested. It is written for project reviewers and assessors, for developers who will maintain or extend the system, and for anyone setting it up on a new machine. Section 10 is a user guide for students.

1.2 The problem

Past examination papers are one of the best ways to prepare for exams, but students often have the paper without a memorandum, or with a memorandum that gives only final answers. Without worked solutions it is hard to understand *how* an answer is reached, so students get stuck on the same kind of question repeatedly. Papers are also usually available only as scanned PDFs or photos, which cannot be searched or copied into other tools.

1.3 Objectives

- Extract the individual questions from an exam paper supplied as a typed PDF, a scanned PDF or a photo.
- Generate, for each question, a concise final answer, an ordered step-by-step solution, an explanation of the underlying concept and exam tips.
- Let students review and correct the extracted questions before solving, so OCR mistakes do not lead to wrong answers.
- Let students save answers they want to revisit and download answers as PDF documents.
- Keep a full history of every paper scanned and every question asked, linked to the student's account.
- Provide the same experience on three platforms: a web app, a desktop app and a mobile app.
- Store user accounts, history and saved answers securely in a MySQL database.

1.4 Scope

In scope	Out of scope (version 1.0)
Account registration and sign-in with email and password	Password reset by email, email verification
OCR of typed PDFs, scanned PDFs and images (up to 30 pages, 20 MB)	Handwritten answer marking
AI question splitting and step-by-step solutions	Rendering of formatted mathematics (LaTeX)
History and saved answers stored per user in MySQL	Offline use without the server
Web, desktop (Electron) and mobile (Expo) apps	Teacher or administrator dashboards
PDF download of answers	Sharing answers with other users

1.5 Definitions

Term	Meaning
OCR	Optical Character Recognition: converting an image of text into machine-readable text.
Paper / session	One uploaded exam paper, or one typed question, together with its questions and solutions. Shown as one entry in History.
Solution	The AI output for one question: answer, steps, explanation and tips.
Saved answer	A solved question the student bookmarked with Save.
JWT	JSON Web Token: a signed token that proves the student is signed in.
API	The StudySync server that all three apps talk to over HTTP.

2. Features

The table below lists the features available to students in version 1.0 and which apps provide them.

Feature	Description	Web	Desktop	Mobile
Register and sign in	Create an account with full name, email and password; stay signed in for 7 days.	Yes	Yes	Yes
Upload a paper	Drag and drop or browse for a PDF or image (PDF, PNG, JPG, WEBP, BMP, TIFF).	Yes	Yes	Yes
Photograph a paper	Take a photo with the phone camera and upload it directly.	-	-	Yes
Type a question	Ask a single question without uploading a paper; it is solved immediately.	Yes	Yes	Yes
Question extraction	OCR plus AI split the paper into numbered questions with marks.	Yes	Yes	Yes
Edit questions	Fix OCR mistakes in any question before solving it.	Yes	Yes	Yes
Solve / Solve all	Solve one question, or all questions (three at a time).	Yes	Yes	Yes
Worked solutions	Answer, numbered steps, explanation and exam tips for each question.	Yes	Yes	Yes
Regenerate	Ask for a fresh solution to a question.	Yes	Yes	Yes
Save answers	Bookmark solved questions; search and remove them later.	Yes	Yes	Yes
History	Every paper and typed question, grouped by day, searchable, reopenable.	Yes	Yes	Yes
Download PDF	Download one answer, a whole paper or all saved answers as a PDF.	Yes	Yes	Share sheet
Sync across devices	History and saved answers are stored in the account, not the device.	Yes	Yes	Yes
Black theme	A pure black interface designed for comfortable long study sessions.	Yes	Yes	Yes

3. System architecture

StudySync follows a client-server architecture. Three client apps share one backend API. The API owns all business logic: authentication, OCR, communication with OpenAI, and reading and writing the database. The clients only display data and send the student's actions to the API.

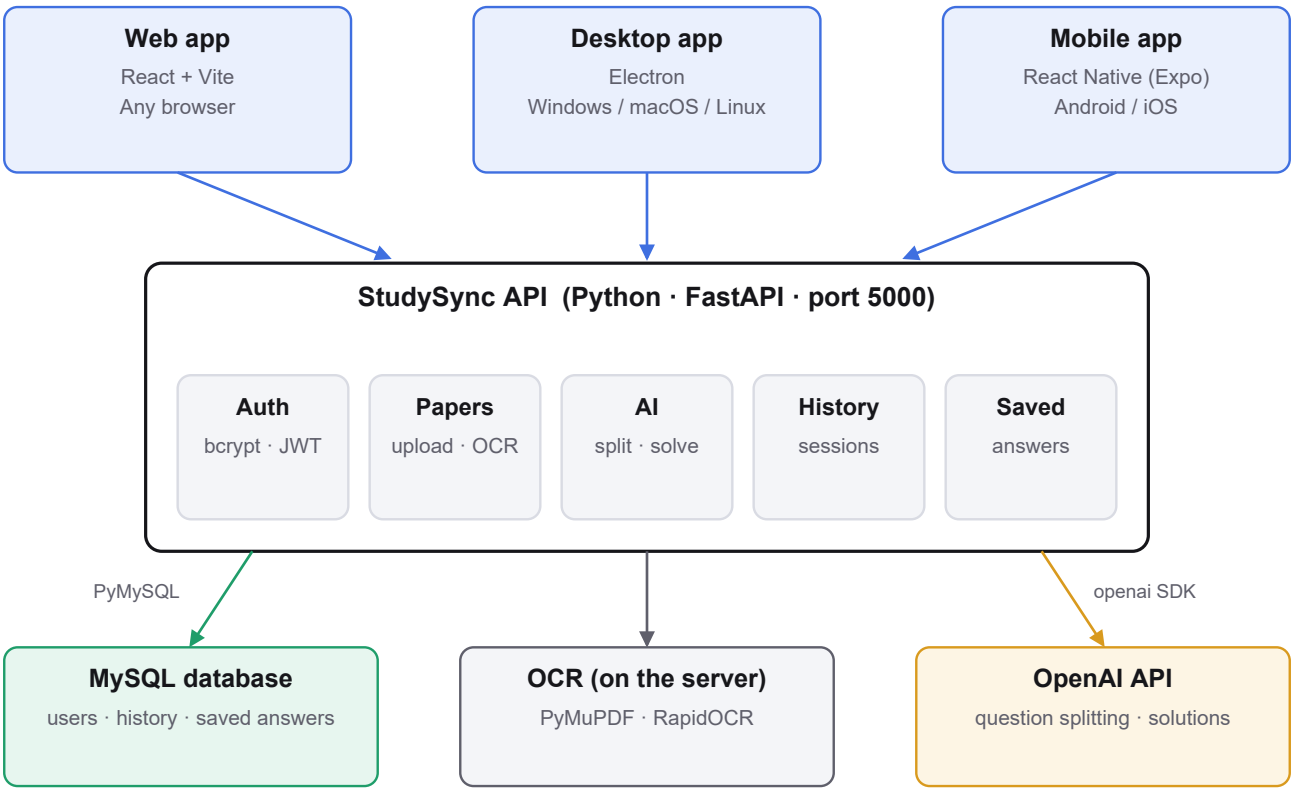


Figure 1. High-level architecture. The apps call the API over HTTP(S) with JSON and a signed-in token (Authorization: Bearer).

3.1 Components

Component	Technology	Responsibility
Web app	React 19, Vite 8, lucide-react icons, jsPDF	Browser interface: sign-in, scanning, solving, history, saved answers, PDF download.
Desktop app	Electron 44, electron-builder	Packages the web app as a native window and installer for Windows, macOS and Linux.
Mobile app	React Native 0.86, Expo SDK 57	Phone interface with camera capture, image and PDF pickers, and PDF sharing.
API server	Python 3.11+, FastAPI, Uvicorn	REST API, validation, authentication, orchestration of OCR and AI.
OCR engine	PyMuPDF, RapidOCR (PP-OCRv6 on ONNX Runtime), OpenCV	Reads text layers of typed PDFs; recognises text in scanned pages and photos.
AI service	OpenAI Chat Completions (JSON mode), default model gpt-4o-mini	Cleans OCR text, splits it into questions, and writes step-by-step solutions.

Component	Technology	Responsibility
Database	MySQL 8+, accessed with PyMySQL	Stores users, history sessions, history questions and saved answers.

3.2 Design decisions

- **One backend for all clients.** All logic lives in the API, so the three apps behave identically and a fix in one place reaches every platform.
- **OCR on the server.** OCR models are large; running them on the server keeps the apps small and gives the same accuracy on every device.
- **Typed PDFs skip OCR.** If a PDF page already contains text, it is read directly. This is faster and exact; OCR is used only where it is needed.
- **AI in JSON mode.** OpenAI is asked to return strict JSON, which the server validates and normalises before any client sees it.
- **Optimistic, queued saving.** The apps update the screen immediately and save to the server in the background, sending only the newest version of each item so slow requests can never overwrite newer data.
- **Desktop reuses the web app.** Electron loads the built web app, so the desktop version needs no separate interface code.

3.3 Technology stack

Layer	Packages and versions used in development
Backend	Python 3.14, FastAPI, Uvicorn, python-multipart, PyMySQL, bcrypt, PyJWT, python-dotenv, openai 3.x, PyMuPDF 1.28, RapidOCR 3.10, ONNX Runtime 1.31
Web	React 19.3, React DOM, Vite 8.3, @vitejs/plugin-react, lucide-react 1.55, jsPDF 4.2
Desktop	Electron 44.7, electron-builder 26
Mobile	Expo 57, React Native 0.86, expo-document-picker, expo-image-picker, expo-print, expo-sharing, AsyncStorage, react-native-safe-area-context, @expo/vector-icons
Database	MySQL 8 or newer (InnoDB, utf8mb4)
Tooling	Node.js 20+, npm, concurrently, Git

4. How it works

This section follows a paper from upload to worked solution, then explains how history and saved answers are kept in sync.

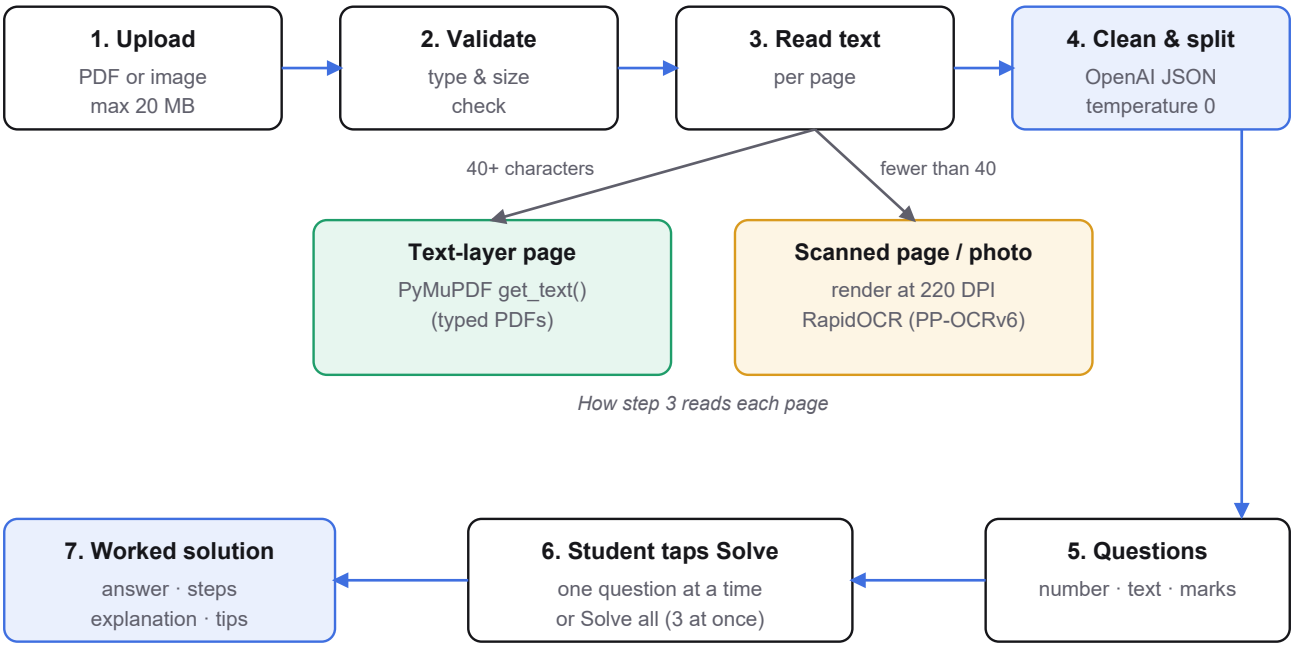


Figure 2. From uploaded paper to worked solution

4.1 Extracting questions

1. **Upload.** The app sends the file to `POST /api/papers/extract`. The server accepts PDF, PNG, JPG, WEBP, BMP and TIFF files up to 20 MB (configurable with `MAX_UPLOAD_MB`).
2. **Read text.** For a PDF, up to 30 pages are processed. Each page's text layer is read with PyMuPDF. A page with fewer than 40 non-space characters is treated as scanned: it is rendered at 220 DPI and passed to RapidOCR. Images go straight to RapidOCR, after being reduced to at most 3000 pixels on the longest side.
3. **Rebuild lines.** RapidOCR returns text boxes. The server groups boxes into lines by vertical position and orders them left to right, which restores the reading order of the page.
4. **Split into questions.** The text (up to 60,000 characters) is sent to OpenAI with instructions to fix OCR errors, keep numbering such as 1.1 or (a), keep mark allocations, include shared context (for example a passage or table) with each sub-question, and ignore cover-page instructions. The response is a JSON object with a title, a subject and a list of questions.
5. **Return.** The server responds with the title, subject, questions, the raw extracted text, the method used (*pdf-text*, *ocr* or *pdf-text+ocr*) and the page count. The app shows the questions and records the paper in History.

4.2 Solving a question

When the student taps **Solve**, the app sends the question text, its marks and the paper's subject to `POST /api/papers/solve`. The server asks OpenAI to act as a patient tutor and to return JSON with four parts:

Part	Content
answer	The final answer, kept short.
steps	An ordered list of steps, each with a title and a detailed explanation, showing the working a student would write in an exam.
explanation	Two to five sentences on the concept behind the question.
tips	Up to three common mistakes or exam tips.

The prompt asks for plain-text mathematics (for example `x^2 + 3x - 4 = 0`) and tells the model to state any assumption when a question is ambiguous, which often happens with OCR text. **Solve all** solves up to three questions at the same time.

4.3 History and saved answers

Every paper and typed question receives an id when it is created. Whenever a question is solved or edited, the app updates the matching history entry on screen and sends the whole session to `PUT /api/history/{id}`, which creates or replaces the session and its questions in one database transaction. Saving an answer calls `POST /api/saved`; removing it calls `DELETE /api/saved`.

Changes are sent through a per-item queue: while a request for an item is in flight, newer changes to the same item wait, and only the newest is sent next. This guarantees that a slow earlier request can never overwrite a later one and that a deleted item cannot reappear. If a save fails, a red banner explains the problem, and the browser warns before the tab is closed while changes are still being saved.

> When a student signs in, the app loads their history and saved answers from the server. Data that an earlier version of the app kept only on the device is uploaded to the account automatically on the first sign-in and then removed from the device.

5. Database design

StudySync uses a MySQL database named `studysync` with four tables. All tables use the InnoDB engine and the utf8mb4 character set, so any language and symbol can be stored. Every row of history and saved answers belongs to a user through a foreign key with `ON DELETE CASCADE`: deleting a user deletes all of their data.

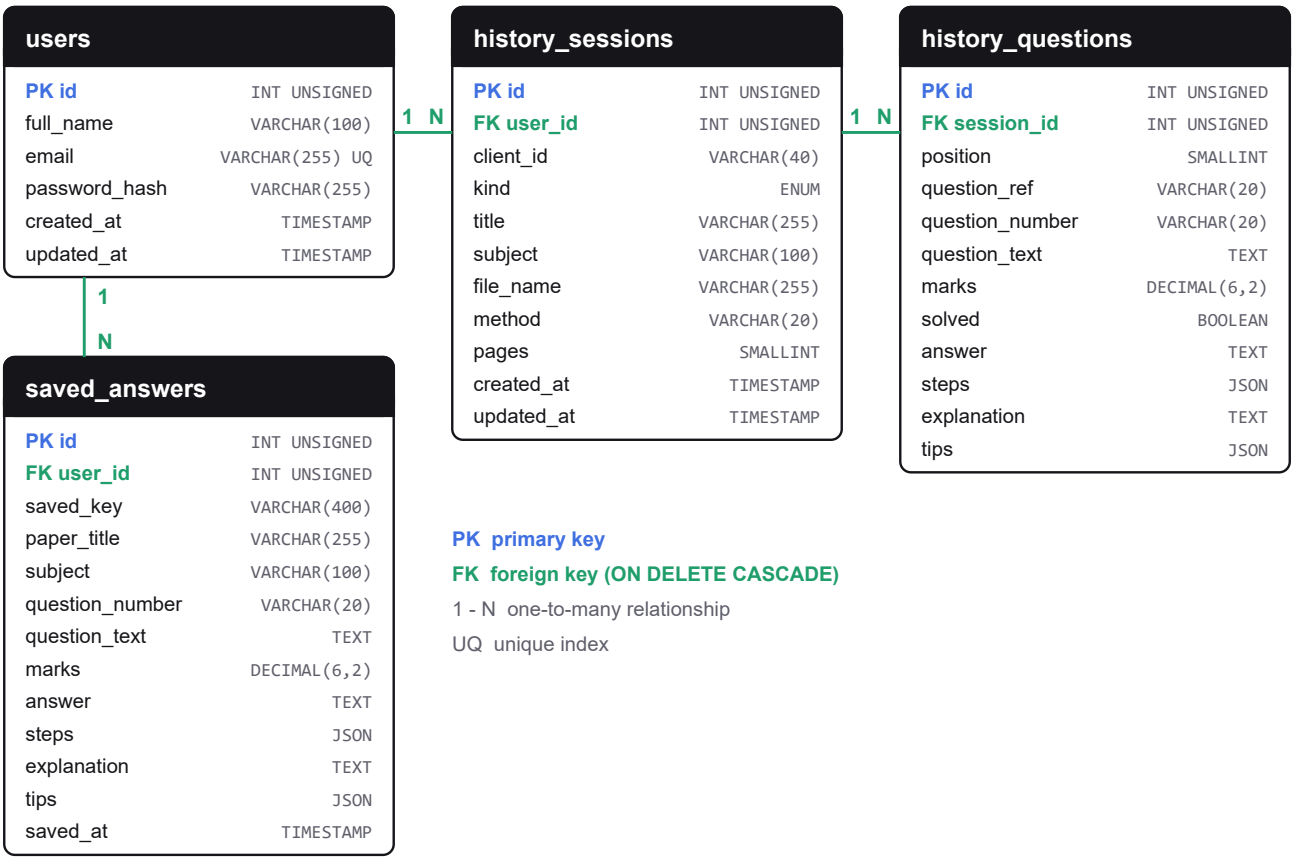


Figure 3. Entity-relationship diagram

5.1 users

One row per registered student.

Column	Type	Notes
id	INT UNSIGNED, PK, auto-increment	Internal user id, used in every other table.
full_name	VARCHAR(100)	Shown in the app header.
email	VARCHAR(255), unique	Stored in lower case; used to sign in.
password_hash	VARCHAR(255)	bcrypt hash (cost 12). The password itself is never stored.
created_at / updated_at	TIMESTAMP	Set automatically by MySQL.

5.2 history_sessions

One row per scanned paper or typed question.

Column	Type	Notes
id	INT UNSIGNED, PK	Internal id.
user_id	INT UNSIGNED, FK to users.id	Owner. Deleting the user deletes the session.
client_id	VARCHAR(40)	Id created by the app (for example h_mv2hk29b_j0hee). Unique per user together with user_id.
kind	ENUM('paper','typed')	Whether the session came from an upload or a typed question.
title, subject	VARCHAR(255), VARCHAR(100)	Detected by the AI from the paper.
file_name, method, pages	VARCHAR, VARCHAR(20), SMALLINT	Original file name, extraction method and page count.
created_at, updated_at	TIMESTAMP (UTC)	History is ordered by updated_at, newest first. The latest 100 sessions per user are kept.

5.3 history_questions

One row per question inside a session, in the order they appear on the paper.

Column	Type	Notes
id	INT UNSIGNED, PK	Internal id.
session_id	INT UNSIGNED, FK to history_sessions.id	Deleting a session deletes its questions.
position	SMALLINT	Order on the paper (0, 1, 2 ...).
question_ref, question_number	VARCHAR(20)	The app's id (q1) and the number printed on the paper (1.2).
question_text	TEXT	The question, including any shared context.
marks	DECIMAL(6,2), nullable	Mark allocation if the paper shows one.
solved	BOOLEAN	True once a solution exists.
answer, explanation	TEXT, nullable	Filled when solved.
steps, tips	JSON, nullable	steps: list of {title, detail}; tips: list of strings.

5.4 saved_answers

One row per answer a student saved. The full solution is copied into the row, so a saved answer stays available even after the history session it came from is deleted.

Column	Type	Notes
id	INT UNSIGNED, PK	Internal id.
user_id	INT UNSIGNED, FK to users.id	Owner.
saved_key	VARCHAR(400)	Identifies the question (paper title, number and start of text). Unique per user, so the same question is saved once.
paper_title, subject, question_number, question_text, marks	various	Copy of the question.
answer, steps, explanation, tips	TEXT / JSON	Copy of the solution.
saved_at	TIMESTAMP (UTC)	Saved answers are listed newest first.

5.5 Indexes and integrity

- `uq_users_email`: one account per email address.
- `uq_history_user_client (user_id, client_id)`: a session id is unique per user; two users can never affect each other's sessions.
- `uq_saved_user_key (user_id, saved_key)`: a question is saved at most once per user.
- `ix_history_user_updated`, `ix_saved_user_date`, `ix_questions_session`: fast loading of a user's lists in order.
- All server connections use UTC (`SET time_zone = '+00:00'`), so stored times are never shifted by the server's local time zone.

6. Backend API reference

The API runs on port 5000 by default. Requests and responses use JSON except the file upload, which uses multipart form data. Every route except register, login and health requires the header `Authorization: Bearer <token>` and only reads or changes the signed-in user's own data. Interactive documentation generated by FastAPI is available at `http://localhost:5000/docs` while the server runs.

6.1 Endpoints

Method	Route	Purpose	Success response
POST	/api/auth/register	Create an account. Body: fullName, email, password (8 to 72 characters).	201 {token, user}
POST	/api/auth/login	Sign in. Body: email, password.	200 {token, user}
GET	/api/auth/me	Return the signed-in user.	200 {user}
POST	/api/papers/extract	Upload a paper (multipart field file) and extract its questions.	200 {title, subject, questions[], rawText, method, pages, fileName}
POST	/api/papers/solve	Solve one question. Body: text (max 8000 characters), subject, marks.	200 {answer, steps[], explanation, tips[]}
GET	/api/history	List the user's history, newest first (max 100).	200 {history[]}
PUT	/api/history/{id}	Create or replace one session with its questions.	204
DELETE	/api/history/{id}	Delete one session.	204
DELETE	/api/history	Delete all of the user's history.	204
GET	/api/saved	List saved answers, newest first.	200 {saved[]}
POST	/api/saved	Save an answer (saving the same id again updates it).	201 {saved}
DELETE	/api/saved?key={id}	Remove a saved answer.	204
GET	/api/health	Report server, database and OpenAI configuration status.	200 {status, database, openai}

6.2 Example: solving a question

```
POST /api/papers/solve
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
Content-Type: application/json

{ "text": "Solve for x: 2x + 6 = 14", "subject": "Mathematics", "marks": 2 }

200 OK
{
  "answer": "x = 4",
  "steps": [
    { "title": "Isolate the x term", "detail": "Subtract 6 from both sides: 2x = 8" },
    { "title": "Divide by the coefficient", "detail": "x = 8 / 2 = 4" }
  ],
  "explanation": "Apply inverse operations to both sides until x is alone.",
  "tips": ["Substitute your answer back to check it."]
}
```

6.3 Example: saving a history session

```
PUT /api/history/h_mv2hk29b_j0hee
{
  "kind": "paper", "title": "Mathematics Paper 1 - November 2024",
  "subject": "Mathematics", "fileName": "maths-p1-2024.pdf",
  "method": "ocr", "pages": 4, "createdAt": "2026-10-10T14:24:44Z",
  "questions": [
    { "id": "q1", "number": "1.1", "text": "Solve for x: 2x + 6 = 14", "marks": 2,
      "solution": { "answer": "x = 4", "steps": [...], "explanation": "...", "tips": [...] } },
    { "id": "q2", "number": "1.2", "text": "Factorise fully: x^2 - 5x + 6", "marks": 3,
      "solution": null }
  ]
}

204 No Content
```

6.4 Errors

Every error is returned as `{"error": "message"}` with a message that can be shown to the student as-is. The main status codes are:

Status	When it happens
400	Invalid input: missing fields, wrong file type, file too large, text too long, invalid values.
401	Not signed in, expired or invalid token, or wrong email or password.
404	Unknown route or account not found.
409	An account with this email already exists.
422	No readable text was found in the uploaded file.
429	The AI service is busy; try again shortly.
502	The AI service returned an error or an unreadable response.
503	The database is unreachable, the OpenAI key is missing or invalid, or the OpenAI account has no credit.
500	Unexpected server error (details are written to the server log, not sent to the client).

6.5 Validation limits

Item	Limit
Upload size / PDF pages	20 MB (configurable) / first 30 pages
Question text sent to solve	8,000 characters
Text sent to the AI for splitting	60,000 characters
Questions per history session	200
History sessions kept per user	100 most recent
Stored question text, answer, explanation	20,000 characters each
Steps / tips per solution	50 / 10
Title / subject / question number	255 / 100 / 20 characters

7. The applications

All three apps use the same pure black design with a blue accent colour, and the same layout: a **Scan** area, a **History** list and a **Saved** list. The screenshots in this section were taken from the running apps.

7.1 Web app

The web app is a single-page React application. It runs in any modern browser and adapts to phone-sized screens.

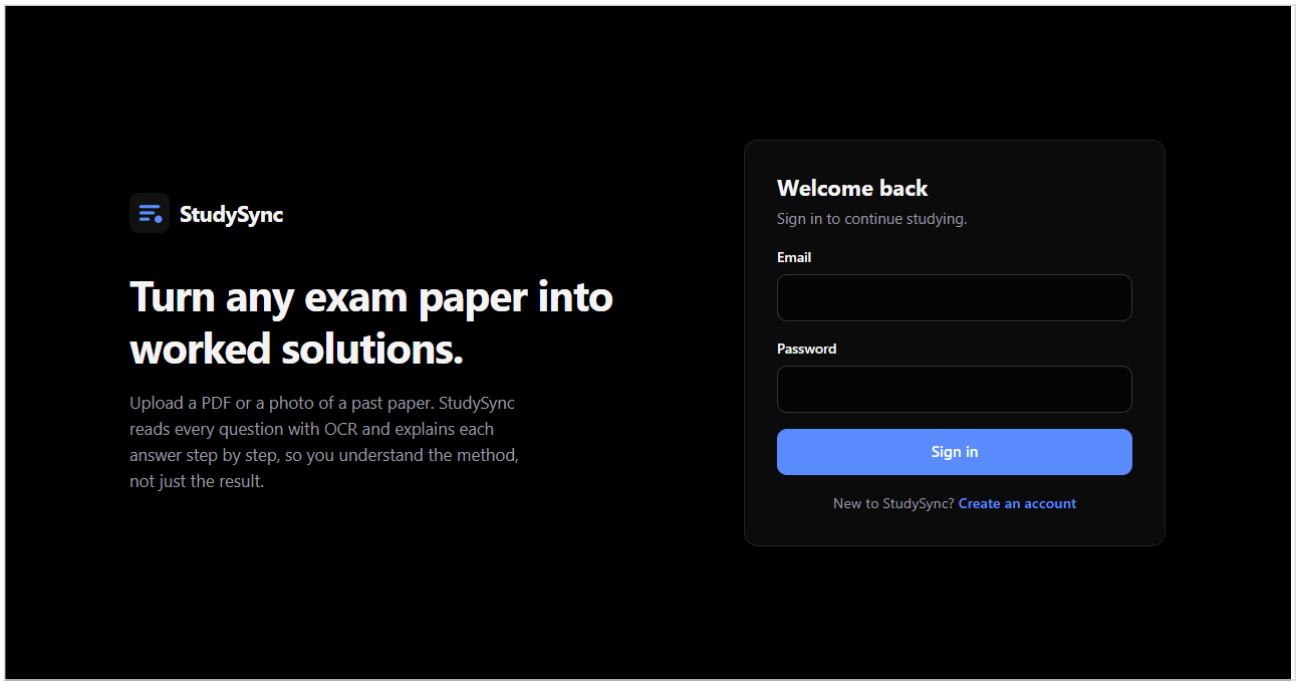


Figure 4. Sign-in page

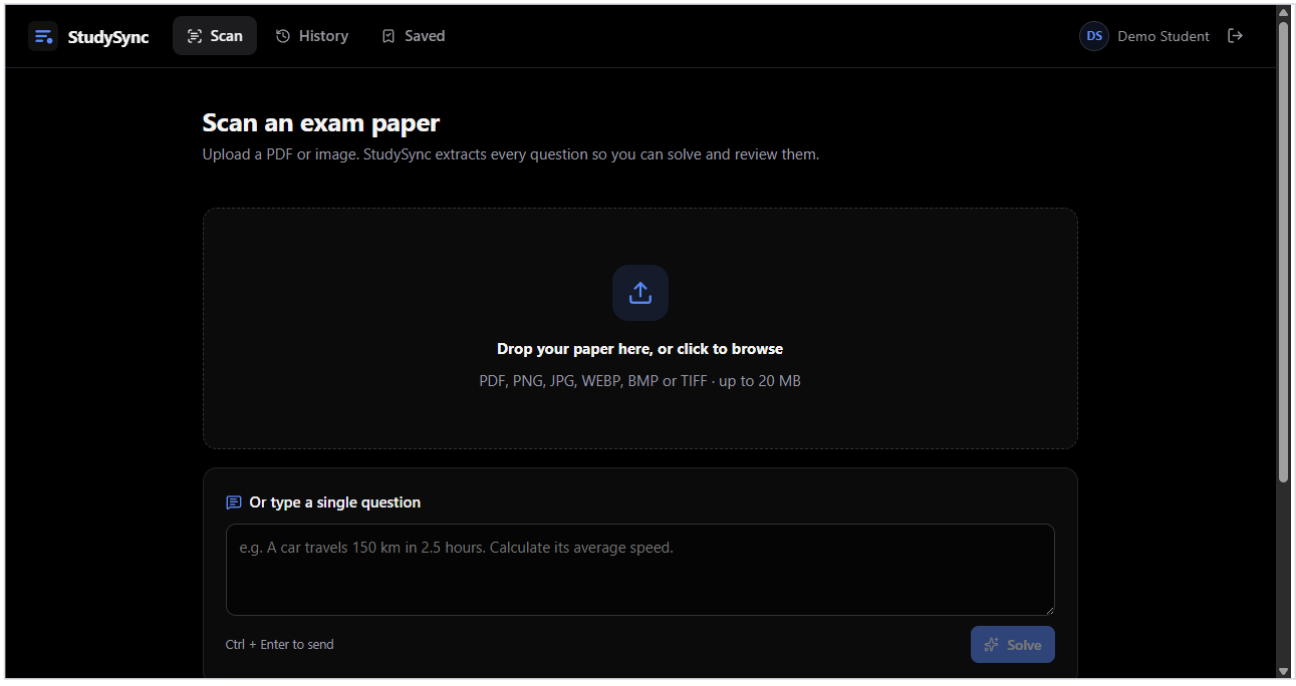


Figure 5. Scan page: upload a paper or type a single question

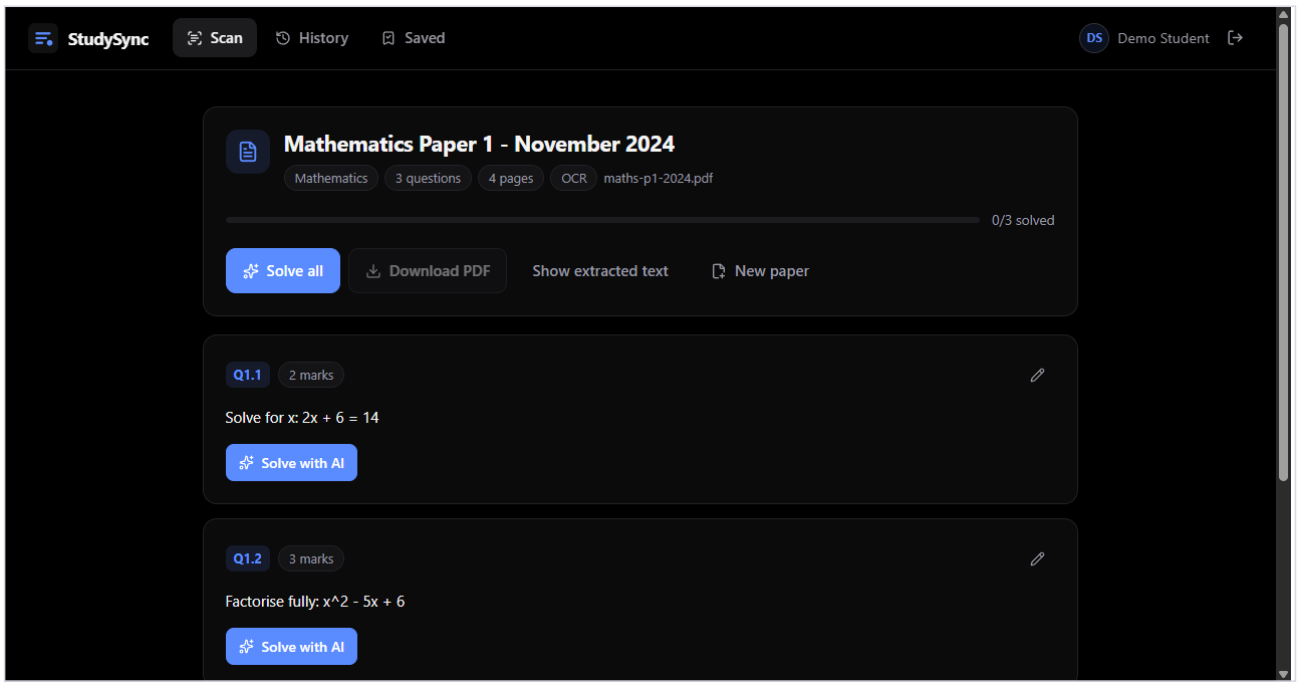


Figure 6. Extracted questions with marks, ready to solve

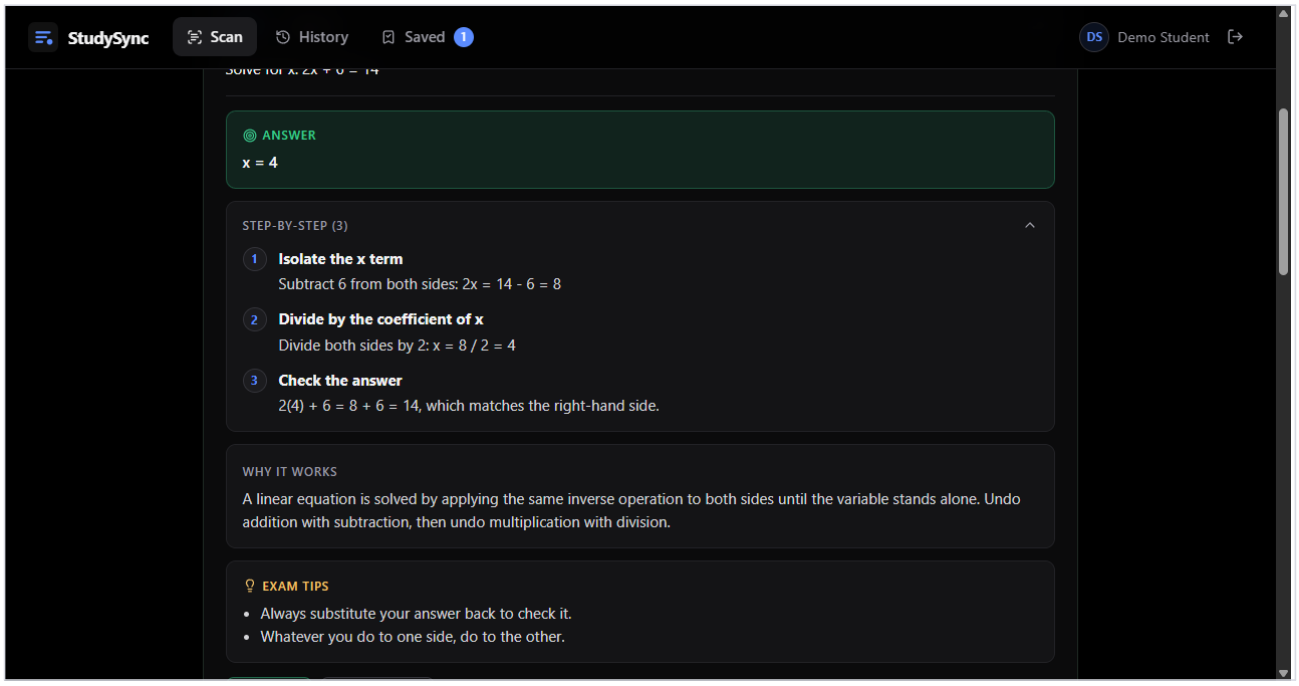


Figure 7. A worked solution: answer, steps, explanation and exam tips

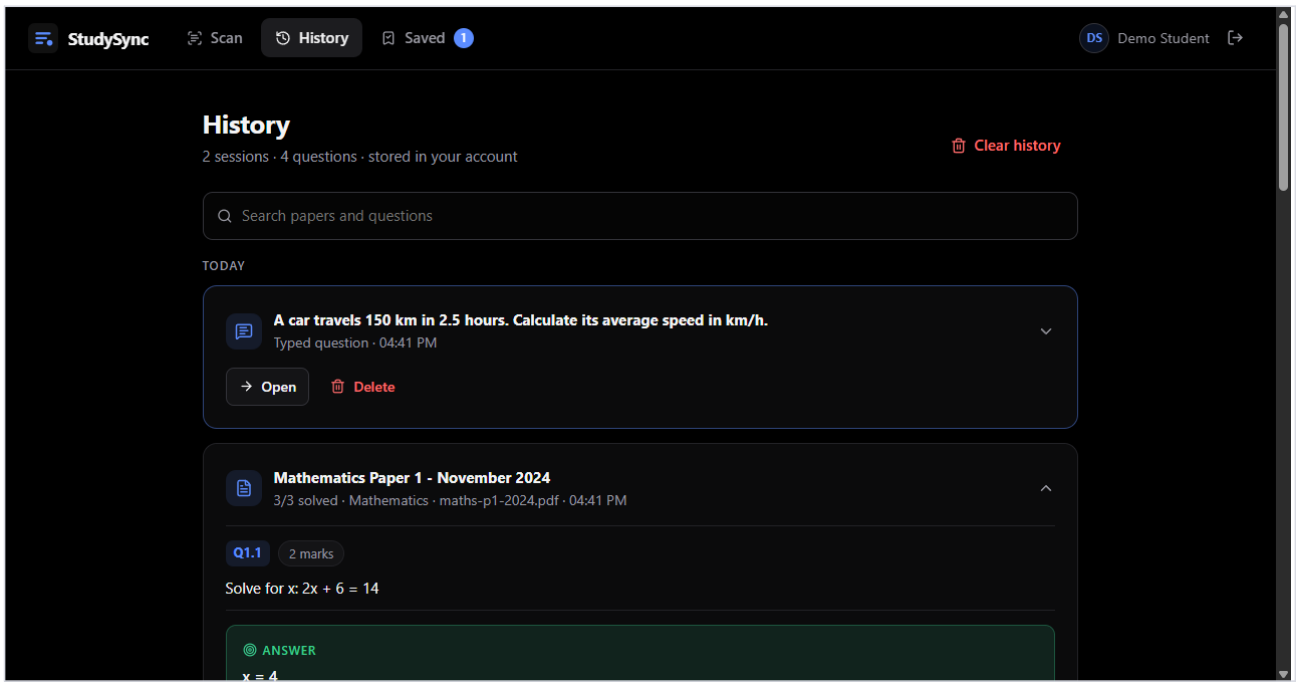


Figure 8. History, stored in the student's account and grouped by day

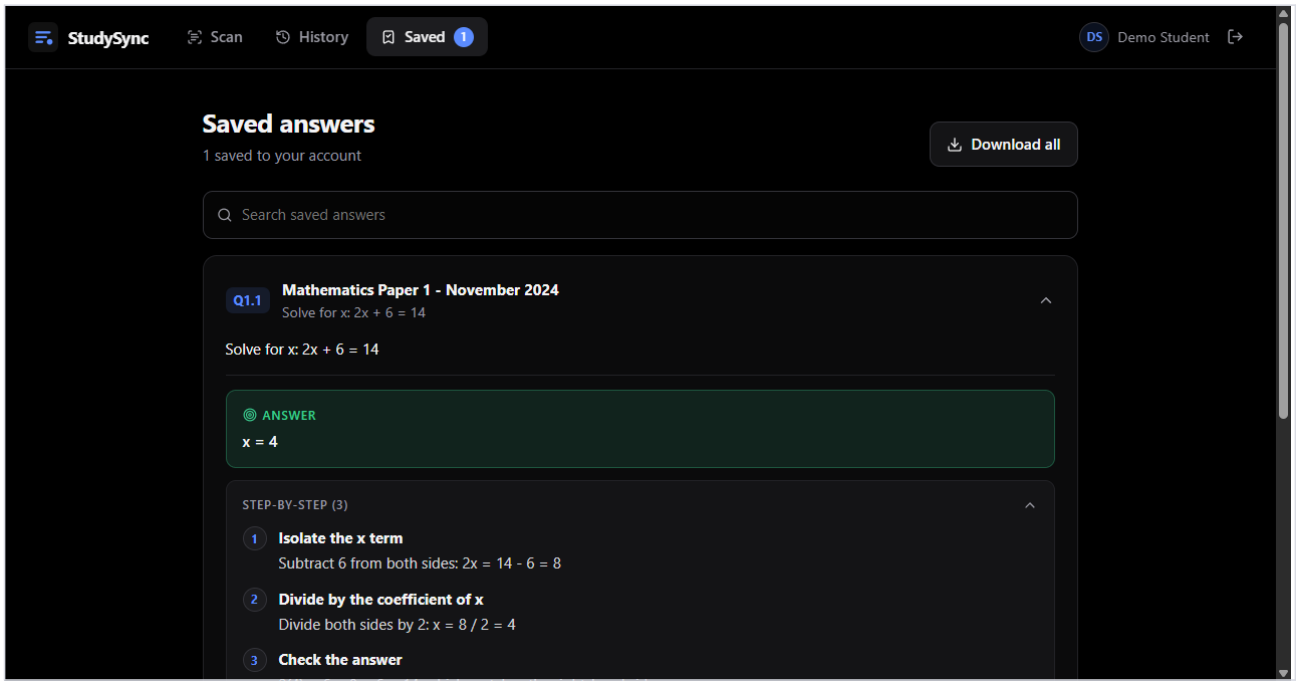


Figure 9. Saved answers with search and PDF download

7.2 Desktop app

The desktop app is an Electron shell that loads the built web app in a native window with a black background. It offers exactly the same features as the web app. External links open in the system browser, and downloaded PDFs are saved through the normal Save dialog. `npm run desktop:dist` produces an installer (NSIS on Windows, DMG on macOS, AppImage on Linux) in `desktop/release/`. The API server must be running and reachable from the computer.

7.3 Mobile app

The mobile app is built with React Native and Expo for Android and iOS. Students can photograph a paper with the camera, choose an image from the gallery or pick a PDF. Answers are turned into a PDF on the phone and shared through the system share sheet, so they can be saved to files, sent by email or opened in another app.

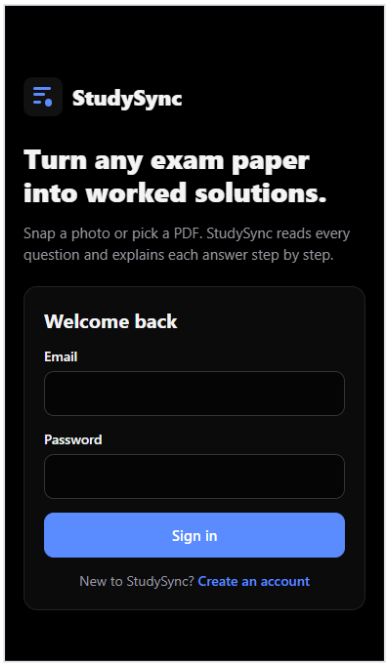


Figure 10. Sign in

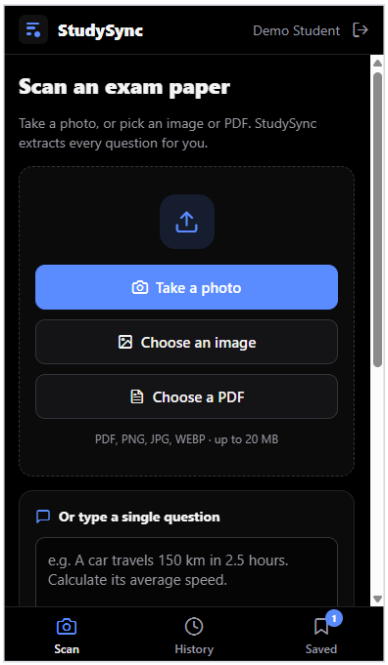


Figure 11. Scan

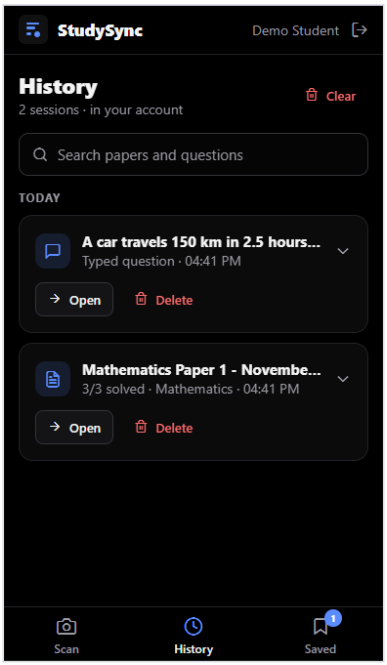


Figure 12. History

> Figure 12 shows the history created in the web app in Figures 7 and 8, loaded on the phone after signing in with the same account. This is the cross-device sync provided by storing history in the database.

8. Security

Area	Measure
Passwords	Hashed with bcrypt (cost factor 12). Passwords must be 8 to 72 characters. Plain passwords are never stored or logged.
Sessions	JSON Web Tokens signed with HS256 using the secret in JWT_SECRET; they expire after 7 days (JWT_EXPIRES_DAYS).
Authorisation	Every data query is filtered by the signed-in user's id. Tests confirm one user cannot read, change or delete another user's history or saved answers.
SQL injection	All database queries use parameterised statements.
Input validation	Request bodies are validated with Pydantic models with length and range limits; uploads are checked for type and size.
Error handling	Internal errors return a generic message; details go only to the server log.
Secrets	The OpenAI key, database password and JWT secret live in server/.env, which is excluded from version control. Example values are in server/.env.example.
Uploaded papers	Processed in memory and discarded; never written to disk or stored in the database.
Data deletion	Students can delete history entries, clear all history and remove saved answers. Deleting a user removes all of their data (cascade).

For a public deployment: serve the API over HTTPS, restrict CORS to the real web address, use a long random JWT_SECRET, create a dedicated MySQL user with access only to the studysync database, and add rate limiting to the sign-in route.

9. Installation and configuration

9.1 Requirements

- Python 3.11 or newer (the backend was developed with Python 3.14).
- Node.js 20 or newer with npm (web, desktop and mobile apps).
- MySQL 8 or newer, for example with MySQL Workbench.
- An OpenAI API key with available credit (platform.openai.com).
- For mobile: the Expo Go app on a phone on the same Wi-Fi network, or an Android/iOS emulator.
- Internet access the first time the server runs, so RapidOCR can download its models if they are missing.

9.2 Install

From the project folder:

```
npm install            # root helper (concurrently)
npm run install:all    # Python virtual environment + web, desktop and mobile packages
```

The server scripts use Windows paths. On macOS or Linux, set up the backend manually:

```
cd server
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

9.3 Configure the server

Copy `server/.env.example` to `server/.env` and fill in the values. Wrap values that contain a # character in double quotes.

```
PORT=5000

DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD="your_mysql_password"
DB_NAME=studysync

JWT_SECRET=a_long_random_string
JWT_EXPIRES_DAYS=7

OPENAI_API_KEY=sk-...
OPENAI_MODEL=gpt-4o-mini

MAX_UPLOAD_MB=20
```

9.4 Create the database

```
npm run db:init                          # uses the settings in server/.env
# or
mysql -u root -p < database/schema.sql
```

`db:init` is safe to run again; it creates missing tables and leaves existing data alone. In MySQL Workbench, refresh the Schemas panel and open **studysync** to browse the tables.

9.5 Run

What	Command	Notes
API and web app	npm run dev	API on http://localhost:5000, web app on http://localhost:5173
API only	npm run server	Same as: cd server, then .venv\Scripts\python run.py --reload
Web app only	npm run web	Requires the API to be running
Desktop app	npm run desktop	Builds the web app, then opens it in Electron
Desktop installer	npm run desktop:dist	Output in desktop/release/
Mobile app	npm run mobile	Scan the QR code with Expo Go

> Start the API with `run.py` (or `npm run server`). Running `uvicorn main:app` from the server folder does not work: the app module is `app.main:app` and the apps expect port 5000, not uvicorn's default 8000.

9.6 Pointing the apps at the API

- **Web and desktop:** set `VITE_API_URL` in `web/.env` (default http://localhost:5000), then rebuild.
- **Mobile:** during development the app automatically uses the computer running Expo on port 5000. To use another server, set `EXPO_PUBLIC_API_URL` in `mobile/.env`. Allow port 5000 through the Windows Firewall so the phone can reach it.
- Check `http://localhost:5000/api/health`: it should report *"database": "ok"* and *"openai": true*.

10. User guide

This section is written for students using StudySync.

10.1 Create an account and sign in

1. Open StudySync and choose **Create an account**.
2. Enter your full name, email address and a password of at least 8 characters, then choose **Create account**.
3. Next time, sign in with your email and password. You stay signed in for 7 days on that device.

10.2 Scan a paper

1. On the **Scan** tab, drag your paper onto the upload area or click it to browse. On a phone, choose **Take a photo**, **Choose an image** or **Choose a PDF**.
2. Wait while StudySync reads the paper. Typed PDFs take a few seconds; scanned papers and photos take longer.
3. Check the extracted questions. If OCR misread something, tap the pencil icon on that question, correct it and choose **Save question**.

10.3 Get solutions

- Choose **Solve with AI** on a question, or **Solve all** to solve the whole paper.
- Read the **Answer**, then work through the numbered **steps**. **Why it works** explains the concept, and **Exam tips** lists common mistakes.
- Not happy with a solution? Choose **Regenerate** for a fresh one.
- To ask one question without a paper, type it in **Or type a single question** on the Scan tab and choose **Solve**.

10.4 Save and download

- Choose **Save** under a solution to keep it in the **Saved** tab. Choose it again to remove it.
- **Download** under a solution creates a PDF of that answer. **Download PDF** at the top of a paper creates a PDF of every solved question. In the Saved tab, **Download all** creates a PDF of all saved answers.
- On a phone, the share sheet opens so you can save the PDF to Files, email it or open it in another app.

10.5 Use your history

- The **History** tab lists every paper and typed question, grouped by day. Use the search box to find a paper or question.
- Expand an entry to read its questions and answers. Choose **Continue** to reopen a paper and solve the remaining questions.
- Choose **Delete** to remove one entry, or **Clear history** to remove everything. Saved answers are kept.
- Your history and saved answers are stored in your account, so they appear on any device where you sign in.

11. Testing

The system was tested at three levels: the OCR engine on its own, the API against the real MySQL database, and complete user journeys through the web and mobile interfaces.

11.1 OCR

Input	Method used	Result
Image of a typed paper (5 lines, questions with marks)	RapidOCR	All lines recognised exactly, in the correct order (about 7 s on the first request, including model loading).
Typed PDF	PyMuPDF text layer	Text extracted exactly; OCR skipped.
Scanned PDF (image-only page)	Render at 220 DPI + RapidOCR	All lines recognised exactly.
Image upload through the API	Full extract route	OCR completed in about 5 s and the request continued to the AI step.

11.2 API and database

An automated script registered two test users and exercised the history and saved-answer routes against the real MySQL database. All data and security checks passed:

- Routes reject requests without a valid token.
- Saving, updating and re-saving a session never creates duplicates; question order, solutions and decimal marks round-trip exactly.
- Timestamps are stored and returned in UTC.
- User B cannot see, modify or delete user A's history or saved answers, even when using the same session id.
- Invalid input (unknown kind, invalid id, saved answer without a solution) is rejected with a clear 400 error.
- Clearing history removes all of a user's sessions and leaves no orphaned question rows.
- Deleting a user removes their history and saved answers (cascade).
- Register, login and /me behave correctly; wrong passwords and duplicate emails are rejected.

11.3 End-to-end

Scenario	Result
Web: register through the form, upload a paper, Solve all, save an answer, ask a typed question	Passed. All three questions stored as solved; saved answer stored.
Web: sign out, reload, sign in again	Passed. History and saved answers reloaded from MySQL.
Web: data left by the earlier device-only version	Passed. Uploaded to the account on sign-in and removed from the browser.

Scenario	Result
Web: delete a history entry	Passed. Row removed from MySQL.
Web: leave a paper while a question is still being solved	Passed. The answer is stored in the correct paper's history.
Mobile: register, ask a typed question, save, reload	Passed. History and saved answer reloaded from MySQL.
Cross-device: create history on web, sign in on mobile	Passed. The same history appears on the phone (Figure 12).
Mobile build: Android bundle and Expo dependency check	Passed. 21 of 21 expo-doctor checks.

During testing the OpenAI account had no remaining credit, so the end-to-end journeys used simulated AI responses for the extract and solve steps; every other part (OCR, authentication, database, sync and interfaces) ran for real. The server correctly detects and reports the missing credit. Live AI output should be verified once credit has been added.

12. Limitations and future work

12.1 Known limitations

- Solving and question extraction need an OpenAI account with credit; without it, those features return a clear error.
- AI answers can occasionally be wrong. Students should treat solutions as study support and check them, especially for long or ambiguous questions.
- OCR accuracy drops with handwriting, poor lighting, skewed photos and complex diagrams; questions that depend on diagrams may be incomplete.
- Mathematics is shown as plain text (for example x^2), not formatted equations.
- The apps need a connection to the API; there is no offline mode.
- There is no password reset or email verification yet.
- Android release builds block plain HTTP, so a production API must use HTTPS.
- Only the first 30 pages of a PDF are processed.

12.2 Future work

- Password reset and email verification.
- Render mathematics with LaTeX (for example KaTeX) and support diagrams in questions.
- Deployment to a cloud server with HTTPS, a managed MySQL database and automated backups.
- Rate limiting and account lockout on sign-in.
- An offline cache so recent history can be read without a connection.
- Teacher accounts to share papers and model answers with a class.
- Progress tracking: topics practised, questions attempted and areas to revise.
- An automated unit and integration test suite run on every change.

13. Troubleshooting

Problem	Cause and fix
"Cannot reach the StudySync server at http://localhost:5000"	The API is not running. Start it with npm run server (or npm run dev) and keep that terminal open.
"Database is unavailable"	MySQL is not running or the DB_ settings in server/.env are wrong. Check the password (quote it if it contains #) and run npm run db:init.
"The server is missing an OpenAI API key"	Set OPENAI_API_KEY in server/.env and restart the server.
"The server's OpenAI account has run out of credit"	Add credit at platform.openai.com/settings/organization/billing. A ChatGPT subscription does not include API credit.
"Could not import module main" when starting the server	Use run.py or npm run server. The module is app.main:app and the port must be 5000.
"No readable text was found"	The scan or photo is too blurry or dark. Retake it flat and well lit, or use a 300 DPI scan.
First upload is slow	RapidOCR loads (and if needed downloads) its models on first use. Later uploads are faster.
Phone cannot connect	The phone and computer must be on the same Wi-Fi, and port 5000 must be allowed through the firewall. Or set EXPO_PUBLIC_API_URL.
Red banner: "Couldn't save your latest changes"	The server could not be reached or the session expired. Check the server, then sign out and sign in again.

Appendix A. Project structure

```
StudySync/
|-- README.md                Setup guide
|-- package.json             Root scripts (dev, server, web, desktop, mobile, db:init)
|-- database/
|   `-- schema.sql           MySQL schema: users, history_sessions, history_questions, saved_answers
|-- server/                  Python FastAPI backend
|   |-- run.py               Starts Uvicorn on PORT (default 5000)
|   |-- init_db.py           Creates or updates the database from schema.sql
|   |-- requirements.txt
|   |-- .env.example         Configuration template (copy to .env)
|   `-- app/
|       |-- main.py          App setup, CORS, error handling, /api/health
|       |-- config.py        Reads server/.env
|       |-- db.py            MySQL queries
|       |-- security.py      bcrypt hashing and JWT
|       |-- errors.py        User-facing error type
|       |-- routes/          auth, papers, history, saved, schemas (validation)
|       `-- services/        ocr.py (PyMuPDF + RapidOCR), ai.py (OpenAI prompts)
|-- web/                     React + Vite web app
|   `-- src/
|       |-- App.jsx          Session, tabs, data loading and background saving
|       |-- components/      AuthScreen, ScanView, QuestionCard, SolutionView, HistoryView, SavedView ...
|       `-- lib/             api, history, saved, sync queue, pdf, legacy import
|-- desktop/                 Electron wrapper (main.js) and installer settings
`-- mobile/                  React Native (Expo) app
    |-- App.js               Session, tabs, data loading and background saving
    `-- src/                 screens, components, api, history, saved, sync, pdf, theme
```

A.1 Environment variables (server/.env)

Variable	Default	Purpose
PORT	5000	Port the API listens on.
DB_HOST / DB_PORT	localhost / 3306	MySQL server address.
DB_USER / DB_PASSWORD	root / (empty)	MySQL credentials.
DB_NAME	studysync	Database name.
JWT_SECRET	(insecure development value)	Secret used to sign sign-in tokens. Must be long and random.
JWT_EXPIRES_DAYS	7	How long a sign-in lasts.
OPENAI_API_KEY	(none)	OpenAI key used for extraction and solving.
OPENAI_MODEL	gpt-4o-mini	OpenAI model name.
MAX_UPLOAD_MB	20	Maximum upload size in megabytes.

A.2 npm scripts (project root)

Script	What it does
install:server	Creates server/.venv and installs Python packages.
install:all	install:server plus npm install for web, desktop and mobile.
db:init	Creates or updates the MySQL database and tables.
server	Starts the API with auto-reload.
web	Starts the Vite development server for the web app.
dev	Starts the API and the web app together.
desktop	Builds the web app and opens the desktop app.
desktop:dist	Builds a desktop installer.
mobile	Starts the Expo development server.